Security self-assessment

LoanReady prototype, FinTechathon 2026 Topic E. Scope: the code in this repository running on one machine with synthetic data and the self-built sandbox. Every test path cited below exists and runs in `python tasks.py test` (paths are checked by `tests/e2e/test_generated_docs.py`); results are in `evidence/test_report.txt`.

(a) Permission-tier implementation

Where: `governance/tiers.py` (tier table and blocked set, read-only), `governance/gate.py` (pipeline), `governance/approvals.py` (tokens), `governance/tickets.py` (execution tickets), `governance/caps.py`, `governance/audit.py`; `adapters/` (the only code that calls sandbox write endpoints); `sandbox/` (server-side checks).

Tier table: generated from the code, docs/generated/tier_table.md. Summary: T0 observe and compute (runs at once); T1 drafts and proposals (nothing leaves LoanReady); T2 sends, payments, cancellations and loan-pack submission (execute only after a human approval bound to the exact parameters); T3 autonomous (disabled); BLOCKED (never executes, whoever asks: pay a new payee, add or change a payee account, accept a loan, change limits, disable the gate or kill switch, edit or delete the log, approve one's own proposal, export raw data to a remote model). Unknown action types fail closed.

Pipeline (`GateService.evaluate`, first failure writes one BLOCKED row and stops):

1. Kill switch (blocks everything except Tier 0).
2. Blocked set or unknown type.
3. Origin and provenance: a request whose origin or provenance is untrusted document text cannot initiate or shape an action.
4. Role rules (owner, bookkeeper, agent, system).
5. Parameter schema.
6. Entity checks against stored records only: the payee exists, the bill is open, the account the document claims equals the payee's stored account (else `ACCOUNT_CHANGED`).
7. Caps per action, per day, per payee per week and per customer for messages.
8. Tier decision: T0 allowed, T1 drafted, T2 a pending proposal.

`approve` checks maker-checker (a human, not the maker; bookkeepers cannot approve payments; the agent never approves) and issues a token bound to the parameters hash, single use, 15-minute expiry. `execute` re-runs the checks, consumes the token and only then mints a signed, single-use execution ticket bound to the outgoing request; the sandbox verifies the ticket, the token scope and the idempotency key on every write.

(b) Known-risk list

Risk	How it would show up	Mitigation in LoanReady	Tests
Prompt injection	Receipt, memo, POS note or invoice text tells the agent to move money	`Untrusted` marker; data tags in prompts; origin and provenance checks in the gate; entities only from stored records; scanner logs every hit	`tests/agent/test_prompts.py`, `tests/integration/test_injection_integration.p`
Excessive agency	Agent acts beyond its role	Tiers in plain code; hard-coded blocked set; Tier 3 disabled; execution tickets	`tests/governance/test_gate_pipeline.py`, `tests/governance/test_tickets.py`, `tests/governance/test_tiers.py`
Fraudulent invoice (changed bank details)	Payment to a fake account	Claimed account compared with the stored payee account; new payees blocked; MockBank checks again server-side	`tests/integration/test_fraud_invoice.py`, `tests/sandbox/test_bank.py`
Wrong numbers	Model miscalculates	All maths in Decimal code; the model only words text; post-checks reject any number not in the facts	`tests/financing/test_irr.py`, `tests/outputs/test_postcheck.py`, `tests/agent/test_explain_postcheck.py`

Risk	How it would show up	Mitigation in LoanReady	Tests
Approval misuse	Approve one thing, execute another; replay a token	Token bound to the parameters hash, single use, 15-minute expiry; maker-checker; draft body hash bound into the approval	`tests/governance/test_approvals.py`, `tests/integration/test_reminders_flow.py`
Runaway actions	Repeated or oversized transfers or messages	Caps; message caps; idempotency keys; kill switch; bank-side limits; undo window before the value date	`tests/governance/test_caps.py`, `tests/sandbox/test_idempotency_opslog.py`, `tests/governance/test_kill_switch.py`, `tests/integration/test_kill_switch_flow.py`
Log tampering	Evidence altered after the fact	Append-only hash chain, no update or delete path; verification on export; independent sandbox chain	`tests/governance/test_audit_log.py`, `tests/sandbox/test_idempotency_opslog.py`
Privacy leakage	Names or accounts sent to a remote model	Data-class policy (bank identifiers and documents never remote); masking; remote opt-in	`tests/agent/test_policy.py`, `tests/agent/test_masking.py`
Credential exposure	Tokens or keys in logs or git	Scoped sandbox tokens; the write token is read in one module; log redaction; `.env` git-ignored	`tests/adapters/test_isolation_static.py`, `tests/governance/test_audit_log.py`
Invented product facts	A wrong rate is quoted	Fixed, dated catalogue; unknown products refused	`tests/financing/test_compare.py`, `tests/financing/test_catalogue_schema.py`

Risk	How it would show up	Mitigation in LoanReady	Tests
Misleading cost presentation	A factor rate of 1.20 looks like "20%"	APR and EAR side by side; factor-rate explanation	`tests/financing/test_disclosure.py`
Unsuitable advice, acting as a broker	Agent recommends or accepts a loan	Neutral ranking by cost; `ACCEPT_LOAN` blocked; the sandbox lender has no accept endpoint; notices	`tests/governance/test_gate_pipeline.py`, `tests/sandbox/test_pos_messaging_lender.py`
Misread receipts	Wrong amount captured	OCR confidence threshold; owner confirms low-confidence and unmatched receipts	`tests/books/test_receipts.py`
Over-trust in forecasts	One number treated as certain	Low/base/high ranges with stated assumptions; going-concern flag	`tests/planning/test_ordering_property.py`, `tests/planning/test_gap.py`
Accounting errors	Statements do not add up	Double-entry validation; identities tested; reconciliation with itemised differences	`tests/books/test_identities.py`, `tests/books/test_reconcile.py`, `tests/integration/test_identities_seeded.py`
LLM outage or bad JSON	A feature breaks	Schema validation, one retry, deterministic fallback, mock mode	`tests/agent/test_validation.py`
Biased readiness score	Personal attributes influence the score	Business-only inputs by type; disclaimer; no credit decision	`tests/financing/test_readiness.py`

(c) Test results summary

- `python tasks.py integrate` runs the import-layer contracts, static checks (no float on money, no wall-clock calls, sandbox write paths only in `adapters/`, no write-token reads elsewhere) and every package's tests on Linux; `python tasks.py test --all` adds the slow scenario and property tests.
- The headless demo checks S1-S10 end to end (demo summary).
- The latest run's counts are in `evidence/test_report.txt` and the controls-to-tests map in the compliance matrix.

(d) Out of scope and residual risks

- **No real bank or money.** The sandbox simulates the bank, POS, messaging and lender; real integrations would need their own security review, credentials management and provider-side controls.
- **No multi-user authentication.** The UI switches between an owner and a bookkeeper role in one session; there are no logins, sessions or device binding. Maker-checker is enforced by role, not by identity proof.
- **Synthetic data, single currency (USD), single business.**
- **Licensing for credit intermediation is not addressed.** LoanReady compares options and submits a pack to a pretend lender; it does not broker, advise on suitability or accept credit.
- **Model quality.** The small local model (`phi3:mini`) is slow on this hardware and often confidently wrong; its category labels are therefore only suggestions the owner confirms. Remote answers depend on the OpenAI model chosen. Every model answer is validated and its numbers post-checked, and the demo and evidence never use a model. The OpenAI budget guard (CTL-AI-07) caps calls and dollars, but its dollar figures are estimates from the prices set in `.env`; OpenAI's own billing is the authority. Payee watchlist screening (CTL-PAY-06) is not built.
- **The approval token and ticket keys live in process memory**; a compromised host could bypass them. Hardware-backed keys and separate processes for the gate would be needed in production.
- **OCR**: receipts are read from OCR sidecar files in this build; Tesseract is not wired.

- Regulatory statements describe design alignment only, never compliance or certification.